

Transition System Modelling

We build a transition system using timestamped JSON files. And use Z3 as the underlying solver to write and verify various LTL properties.

JSON → Symbolic transition system

Below is the pipeline of the same.

Data Cleaning

1. Primary key

The file `swerve_sim1.json` has multiple top level keys. The core idea is to find the key with the longest records.

`discover_time_series()` : Finds the key with the largest number of records. In case of *swerve* data it is *ground_truth_kinematics*.

2. Remaining keys

The remaining keys are entries with timestamps. We map those to nearest entry in the *primary_key* (based on timestamps of the primary key).

`nearest_by_time()` : For each entry in the primary key find the nearest entry in the remaining keys

(`ego_estimated_kinematic` (480), `perception_objects` (481), `planning_trajectory` (481), and `control_cmds` (1,636) by binary search in *swerve* file).

3. Flatten Records

Recursively flatten JSON objects into key-value pairs. This is done in module `extract_flat_trace()` which flattens the keys into a list of states.

Example

Given `t=138.359` ,

Find nearest points in other keys after deducing the primary key.

```
groundtruth_kinematic  t=138.359 ← this IS the primary frame
ego_estimated_kinematic t=138.326 (Δ=33ms, binary search)
control_cmds           t=138.368 (Δ=9ms)
perception_objects     t=138.389 (Δ=30ms)
planning_trajectory    t=138.410 (Δ=51ms)
```

This is done via *binary search*.

Now, flatten each key into a single dict (representing one state). Example,

```
state[500] = {
  "timestamp": 138.359,
  "meta.swerve_point.x": 81649.492,
  "groundtruth_kinematic.groundtruth_ego.pose.position.x": 81555.3,
  "ego_estimated_kinematic.pose.position.x": 81555.1,
  "control_cmds.lateral.steering_tire_angle": -0.002,
  "perception_objects.objects[0].pose.position.x": 81623.7,
```

```
"planning_trajectory.points[0].longitudinal_velocity": 7.23,
}
```



Note: There is a flag `max_list_item` passed to the module `flatten_record()`. The default value is False. This determines the maximum number of elements to fetch from a list while recursively building the object.

Key observations

1. The values `"123 2 345"` are converted to a list of floats `[123, 2, 345]`.
2. No keys are recorded when the list is empty.
3. This will explode as the number of objects in a list might be a lot. Hence a later improvement could be use points at interval differences of 25%.

Data profiling

The states are then profiled to add statistics like `max`, `min` `ego_position.x`. Example,

Variable (shortened)	min	max	Δ min	Δ max	initial	flags
<code>ego.position.x</code>	81,547.42	81,696.59	-0.006	0.644	81,547.42	—
<code>groundtruth.position.x</code>	81,547.51	81,696.57	0.000	0.141	81,547.51	monotone ↑
<code>perception.position.x</code>	0.0	81,647.94	-81,615	+81,613	0.0	—
<code>control.lateral.time</code>	125.86	173.81	0.000	0.113	125.86	monotone ↑
<code>perception.classification.label</code>	0.0	1.0	—	—	0.0	boolean

This is done in `profile_trace()` module.

Build Z3 transition system

Given the flattened trace from the above processes. This step builds a z3 symbolic model. This step iterates over all non-constant variables and generates atomic propositions.

- Partition variables of state into two categories : *constants* and *state variables*.
 - Example,


```
ego_estimated_kinematic.acceleration.angular.x = 0.0 (ego never rotates)
```

 This is stored as a constant in `self.constants`.
 Other variables which change are stored in the state dicts. This is to optimize space.
- Convert the reals in the states to `Real` z3 objects.

1. `make_state(0)` :

Convert a state dict to equivalent z3 expressions. Example,

```
s0["control_cmds.lateral.steering_tire_angle"] →
  Real("control_cmds_lateral_steering_tire_angle_0")

s1["control_cmds.lateral.steering_tire_angle"] →
  Real("control_cmds_lateral_steering_tire_angle_1")
```

2. `build_propositions`

Iterates over non-constant variables of all states and make atomic propositions based on the them.

Case 1: Boolean variables

For variables like `perception_objects.objects[0].classification[0].label` that only take values `{0.0, 1.0}`:

```
_true  → λ s: s[var] >= 0.5    (is it 1?)
_false → λ s: s[var] < 0.5    (is it 0?)
```

Case 2: Numerical variables

For every numeric variable, two threshold propositions based on the quartiles observed in the data:

```
_high → λ s: s[var] > q75    (above 75th percentile)
_low  → λ s: s[var] < q25    (below 25th percentile)
```

For example, `control_cmds.lateral.steering_tire_angle` has `q25=-0.001`, `q75=-0.0`:

```
steering..._high → s[var] > -0.0    (steering more right than usual)
steering..._low  → s[var] < -0.001  (steering more left than usual)
```

If a variable's observed range includes 0 (i.e., `min_val ≤ 0 ≤ max_val`), an extra proposition:

```
_near_zero → λ s: And(s[var] >= -ε, s[var] <= ε)
              where ε = max(range * 5%, 0.01)
```

This fires for steering when it's essentially straight, for velocity when nearly stopped, etc.



Note : The states store both real values and such propositions. Need to add a flag to disable such proposition generation.

Default Constraints

1. Initial constraint : by default `exact_initial` flag is set to **True**. Otherwise, constraints such as `initial_position.x +- delta` is added to the propositions. This allows noise.
2. Invariant constraints : uses a flag `use_invariants` to enable or disable invariant properties. Example,

```
ts.add_invariant(lambda s: s["ego_estimated_kinematic.twist.linear.x"] >= 0)
```

Model Checking

Now that a transition system has been generated, **LTL** properties can be verified on both.

Over propositions:

```
ts.ltl_G(path, ts.get_prop("steering_tire_angle_high"))
# → And(steering_0 > q75, steering_1 > q75, ..., steering_k > q75)
```

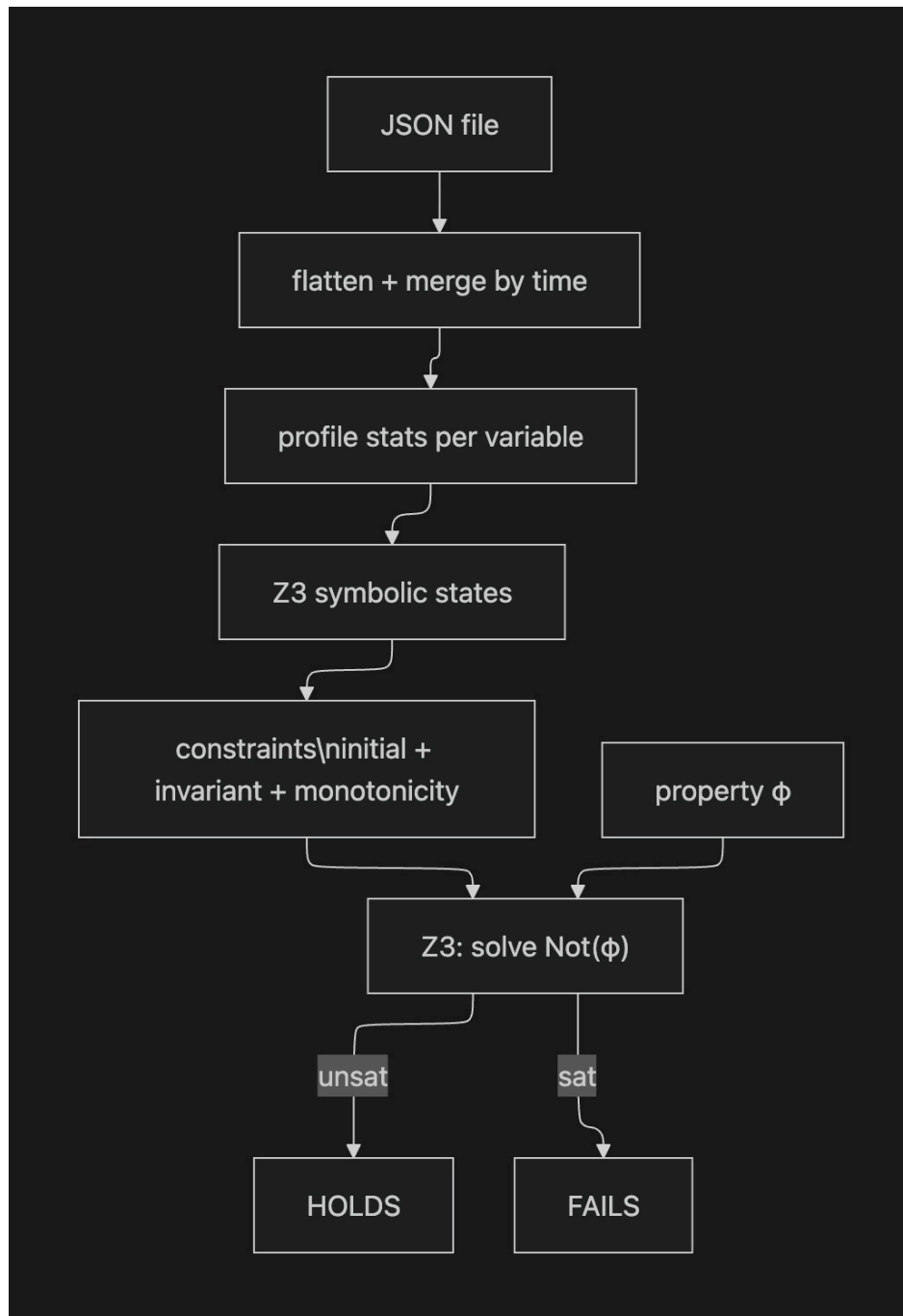
Over raw ArithRef states:

```
ts.ltl_G(path, lambda s: s["ego_estimated_kinematic.pose.position.x"] > 81600.0)
# → And(ego_position_x_0 > 81600, ego_position_x_1 > 81600, ..., ego_position_x_k > 81600)
```

Available LTL functionalities:

- $G(p)$ — p at every step: `And(p(s0), p(s1), ..., p(sk))`
- $F(p)$ — p at some step: `Or(p(s0), p(s1), ..., p(sk))`
- $G(p \rightarrow q)$ — whenever p , also q : `And(Implies(p(si), q(si)) for all i)`
- $G(p \rightarrow Fq)$ — whenever p , eventually q : for each i where $p(s_i)$, require `Or(q(sj) for j ≥ i)`
- $\neg p \cup q$ — p stays false until q fires

Flowchart



Properties verified

Below are the LTL properties based on `swerve` data we have verified so far.

1. $G(\text{var} \in [\text{min}, \text{max}])$ — Safety

```

ts.check(
  f"G({short} ∈ [{vp.min_val:.1f}, {vp.max_val:.1f}])",
  lambda p, n=name, lo=vp.min_val, hi=vp.max_val:
    ts.ltl_G(p, lambda s, n=n, lo=lo, hi=hi:
      And(s[n] >= lo, s[n] <= hi)),
  bound,
)

```

2. $G(\text{var}[t+1] \geq \text{var}[t])$ — Monotone increasing

```
ts.check(  
    f"G({short}[t+1] ≥ {short}[t]) – monotone increasing",  
    lambda p, n=name: And(*[  
        p[i + 1][n] >= p[i][n] for i in range(len(p) - 1)  
    ]),  
    bound,  
)
```

3. $G(\text{var}[t+1] \leq \text{var}[t])$ — Monotone decreasing

```
ts.check(  
    f"G({short}[t+1] ≤ {short}[t]) – monotone decreasing",  
    lambda p, n=name: And(*[  
        p[i + 1][n] <= p[i][n] for i in range(len(p) - 1)  
    ]),  
    bound,  
)
```

4. $F(\text{var})$ — Liveness

```
ts.check(  
    f"F({short}) – eventually true",  
    lambda p, n=name: ts.ltL_F(p, lambda s, n=n: s[n] >= 0.5),  
    bound,  
)
```

5. $\exists t: \text{var}[t] \geq \text{max}$ — Reachability

```
ts.check_reachable(  
    f"Can {short} reach its max ({vp.max_val:.2f})?",  
    lambda p, n=name, mx=vp.max_val:  
        Or(*[p[i][n] >= mx for i in range(len(p))]),  
    bound,  
)
```